12.1.4. The module cache

Modules can be loaded by specifying the location of a file, but they're not loaded from source each time they're required. Modules are cached, meaning that the second, third, or other subsequent load of the same file in your application will *not* evaluate the file again. Instead, loading any module again anywhere else in the application will return the *same object* with the same properties assigned to `exports` the first time the file was evaluated.

Why does this matter? Suppose you want to keep track of all the posts created inside your post module:

```
class Post
  posts = []
  constructor: (@title, @body) ->
    posts.push @
  @all: -> posts

exports.Post = Post
```

It doesn't matter how many times you require it; the same object will be returned by the `require` method:

```
{Post} = require './post'

aPost = new Post 'A Post', 'Some content'
anotherPost = new Post 'Another Post', 'Some more content'
Post.all().length
# 2
```

When you `require` the same module again (in the same REPL session or program), you get a reference to the same object, even if you assign it to a different name:

```
TheSamePost = require('./post').Post

aThirdPost = new Post 'Three', 'Content three'
TheSamePost.all().length
# 3
```